# UNIT -II

## ❖ Class

In Java, a class is a blueprint or template for creating objects. It defines the structure and behavior of objects that will be created based on that class. A class typically includes fields (also called member variables or instance variables) to store data and methods (functions) to define the behavior or operations that can be performed on objects of that class.

Syntax:

```
accessModifier class ClassName {
 // Fields (variables)
 dataType fieldName1;
 dataType fieldName2;
 // Constructors
 ClassName(parameters) {
 // Constructor code
 }
```

- accessModifier: Specifies the visibility of the class, which can be public, private, protected, or package-private (no modifier).
- class: The keyword used to declare a class.
- ClassName: The name of the class, following Java naming conventions (starts with an uppercase letter).
- dataType: The data type of fields and return types of methods.
- fieldName1, fieldName2: Names of fields.
- Constructor: Special methods with the same name as the class, used for initializing objects.
- methodName1, methodName2: Names of methods.

## ❖ Object

In Java, an object is an instance of a class. Objects are used to represent real-world entities and their interactions. Each object is created based on a class, and it contains data (attributes or fields) and methods (functions) defined in that class.

Syntax of Creating an Object:

To create an object in Java, you follow this syntax:

ClassName objectName = new ClassName();

- ClassName: Replace this with the name of the class from which you want to create an object.
- objectName: Choose a name for the object. This is the reference variable that allows you to access the object.
- new: The new keyword is used to create a new instance of the class.

## ❖ Using predefined classes

Using predefined classes in Java involves leveraging the built-in classes and libraries provided by the Java standard library (Java API) to perform various tasks without having to implement everything from scratch.

1. Importing Classes: To use predefined classes, you import them using the import statement at the beginning of your Java file. For example:

```
import java.util.ArrayList;

import java.io.File;
```

2) Common Predefined Classes

1. java.lang: This package is automatically imported and contains fundamental classes like String, System, and Math.
2. java.util: Provides classes for data structures (e.g., ArrayList, HashMap) and utilities (e.g., Scanner, Random).
3. java.io: Offers classes for input and output operations, including file handling (e.g., File, InputStream, OutputStream).
4. java.time: Includes classes for working with dates and times (e.g., LocalDate, DateTimeFormatter).
5. java.net: Contains classes for networking tasks (e.g., Socket, URL).

Using Math class

```
double radius = 5.0;
double area = Math.PI * Math.pow(radius, 2);
System.out.println("Area of a circle with radius " + radius + " is: " + area);
```

## ❖ Constructors:

Constructor is specialized method that is used at the time of object creation.
The constructor is called when an object of a class is created. It can be used to set initial values for object attributes.

**Use:**
1. To initialize object/data members/variables
2. To copy/load all members of class into object-> when we create an object.

**Syntax:**

```
Accessmodifier Constructorname()
{
  //code to initialize data members
}
```

**Rules to create constructor:**
1. Name of constructor and classname should be same.
2. constructor can't have returntype.

Why constructor don't have returntype?
Constructor is getting called automatically while creating object so there is no need to have returntype for constructor.

3. In single class you can have multiple constructors name of constructor should be classname but arguments should be different.

4. constructor can accept arguments.
5. Constructor has access specifiers/modifiers as private, public, default and protected.

Example:

```
package constructor;
public class example1
{
        String name;//global variable
        int id;
        long accountno;
        int atmno;

        public example1(String name, int id, long accountno, int atm)//local variable
        {
          this.name=name;
        // this keyword refers to the current object in a method or constructor.
          this.id=id;
          this.accountno=accountno;
          atmno=atm;
          System.out.println(name+"   "+id+"   "+accountno+"   "+atmno);
        }

        public static void main(String[] args)
        {
         example1 ex1= new example1("Riya",101,45698l,5845);
         //example1-> classname-> datatype
         //ex1->objectname->identifier of your object
         //new-> keyword-> to create empty object in heap area memory
         //example1(....)-> construcotr-> to initialize object/copy all members of class in empty
object

         example1 ex2= new example1("Priya",102,456982l,5846);
        }}
```

**Types of constructors:**

**1. Default constructor:**

-If Constructor is not declared in java class, then at the time of compilation compiler will provide Constructor for the class

- If programmer has declared the constructor in the class, then compiler will not provide default Constructor.

-The Constructor provided by compiler at the time of compilation is known as Default Constructor.

```
package constructor;


public class example2
{
	int a;
	int b;
	int s=10;
	String str="hello";


//	Default constructor created by compiler
//	public example2()
//	{
//		super();//It is used to call superclass methods, and to access the superclass constructor.
//	}



	public static void main(String[] args)
	{
		example2  ex= new example2();//calling Default constructor


	}
}
```

**1.User Defined constructor without parameters( no-argument constructor)**

Syntax:

Accessmodifier constructorname()
{
  //code to initialize global variables
}

```
package constructor;

public class example4
{

	int n;

	//user defined constructor without parameters
	example4()
	{
		n=10;
	}

	public void m1()
	{
		System.out.println(n);
	}


	public static void main(String[] args)
	{
		example4 ex= new example4();
		ex.m1();

	}
}
```

**2. User Defined constructor with parameters (Parameterized constructor):**
Syntax:

Accessmodifier constructorname(parameter list with datatype)
{
  //code to initialize global variable
}
To call parameterized constructor
Syntax:
Main()
{
  Abc  obj= new Abc(values of parameters);
}

```
package constructor;
```

```
public class example5
{
   //global variables
            int roll;
            String name;
            String address;
            long mobile;
            double percent;

            //create parameterized constructor
            example5(int a,String b, String d, long m, double p)//local variable
            {
                        roll=a;
                        name=b;
                        address=d;
                        mobile=m;
                        percent=p;
            }

            public void displayinfo()
            {
                        System.out.println("Student information:");
                        System.out.println("Roll no: "+roll);
                        System.out.println("Name of student: "+name);
                        System.out.println("Address: "+address);
                        System.out.println("Mobile No.: "+mobile);
                        System.out.println("percentage: "+percent);
            }
            public static void main(String[] args)
            {
                        //create object
                        example5 obj= new example5(1,"Sachin","Pune",9586852315l,75.02);
                        obj.displayinfo();

                        example5 obj1= new example5(2,"Priya","Mumbai",9586852345l,75.70);
                        obj1.displayinfo();

                        example5 obj2= new example5(3,"Pravin","Pune",9586852320l,85.02);
                        obj2.displayinfo();
   example5 obj4=new example5();
            }
}
```

## ❖ Constructor overloading:

-It is technique which allow for single class we can have multiple constructors which has same name as classname but it must have different type and different number of arguments( signature should be different).

**- Constructor overloading in Java** means to define multiple constructors of a class but each one must have a different signature.

- Java compiler differentiates these constructors based on the number of the parameter lists and their types. Therefore, the signature of each constructor must be different.
- The signature of a constructor consists of its name and sequence of parameter types.
-Constructor overloading is used because it allows initializing objects with different types of data.

```
package constructor;


public class constructoroverloading
{
	int length;
	int breadth;
	int side;
	float radius;

	public constructoroverloading()
	{
		System.out.println("No argument constructor called");
	}

	public constructoroverloading(int length, int breadth)
	{
		this.length=length;
		this.breadth=breadth;
		//System.out.println("Area of rectangle: "+(length*breadth));
	}

	public constructoroverloading(int side)
	{
		this.side=side;
	}
```

```
    public constructoroverloading(float radius)
    {
        this.radius=radius;
    }

    public void arearect()
    {
        System.out.println("Area of rectangle: "+(length*breadth));
    }

    public void areasquare()
    {
        System.out.println("Area of square: "+(side*side));
    }

    public void areacircle()
    {
        System.out.println("Area of circle: "+(3.14*radius*radius));
    }

//  public constructoroverloading(int length,int breadth, int side, int radius)
//  {
//      this.length=length;
//      this.breadth=breadth;
//      this.side=side;
//      this.radius=radius;
//  System.out.println("Area of rectangle: "+(length*breadth));
//  }
    public static void main(String[] args)
    {
```

```
        constructoroverloading obj= new constructoroverloading();

        constructoroverloading obj1= new constructoroverloading(20,30);
        obj1.arearect();

        constructoroverloading obj2= new constructoroverloading(20);
        obj2.areasquare();

        constructoroverloading obj3=new constructoroverloading(3.5f);
        obj3.areacircle();
    }
```

❖ **Constructor chaining:**

1. We can call any constructor in any other constructor.
2. To call we have to use “this” keyword.
3. call to constructor depend on signature of constructor

Syntax:

```
this(value according to signature);
```

ex:

(int)-> this(20);

()-> this();

(int, String)-> this(20,”hello”);

4. constructor call within another constructor must in first statement.
5. within one constructor you can use this keyword one time only for calling constructor.

package constructor;

```
public class constructorchaining
{
        constructorchaining()// default access specifier
```

```
    {
      System.out.println("constructor 1 is called");
    }

    public constructorchaining(int a)
    {
      this();
      System.out.println("constructor 2 is called");
    }
    public constructorchaining(float a)
    {
      this(12);
      System.out.println("constructor 3 is called");
    }
    public static void main(String[] args)
    {
      constructorchaining obj= new constructorchaining(20.3f);

    }
}
```

## ❖ Static Data Members and Methods:

In Java, static members are those which belongs to the class and you can access these members without instantiating the class.

The static keyword can be used with methods, fields, classes (inner/nested), blocks.

### Static Methods –

Method name followed by static keyword is called as static method

Static method is Class level method where object creation not required.

-Memory allocation->task perform->output->memory deallocate

-Static method memory allocates at time of class loading and memory deallocation at the time of class unloading.

**Syntax:**

```
Accessspecifier static returntype methodname(parameters)
{
        //statements
}
```

```
public class MyClass {
  public static void sample(){
    System.out.println("Hello");
  }
  public static void main(String args[]){
    MyClass.sample();
  }
}
```

**Static Members/fields** – You can create a static field by using the keyword static. The static fields have the same value in all the instances of the class. These are created and initialized when the class is loaded for the first time. Just like static methods you can access static fields using the class name (without instantiation).

```
class Student{
  int rollno;//instance variable
  String name;
  static String college ="ITS";//static variable
  //constructor
  Student(int r, String n){
  rollno = r;
  name = n;
  }
  //method to display the values
  void display (){System.out.println(rollno+" "+name+" "+college);}
}
```

```
//Test class to show the values of objects
public class TestStaticVariable1{
 public static void main(String args[]){
 Student s1 = new Student(111,"Karan");
 Student s2 = new Student(222,"Aryan");
 //we can change the college of all objects by the single line of code
 //Student.college="BBDIT";
 s1.display();
 s2.display();
 }
}
```

## ❖ Inner Class:

Java inner class or nested class is a class that is declared inside the class or interface.
We use inner classes to logically group classes and interfaces in one place to be more readable and maintainable.

**Syntax of Inner class**

```
class Java_Outer_class{
 //code
 class Java_Inner_class{
  //code
 }
}
```

**Types of Inner Classes**

There are basically four types of inner classes in java.

1. Nested Inner Class
2. Method Local Inner Classes
3. Static Nested Classes
4. Anonymous Inner Classes

1. **Nested Inner Class:**

   It can access any private instance variable of the outer class. Like any other instance variable, we can have access modifier private, protected, public, and default modifier. Like class, an interface can also be nested and can have access specifiers.

   Example:

```
package Inner_Class;

public class NestedInner {

        class Inner{
          public void show()
          {
                  System.out.println("nested class method");
          }

        }
        public static void main(String args[])
        {
          NestedInner.Inner in =new NestedInner().new Inner();
          in.show();
        }
}
```

2. **Method Local Inner Classes:**

   Inner class can be declared within a method of an outer class which we will be illustrating in the below example where Inner is an inner class in outerMethod().

   Example:

```
package Inner_Class;

public class MethodLocalInnerClasses {

void outermethod()
{
System.out.println("inside outer method");

class Inner
{
void innermethod()
{
System.out.println("inside inner method");
}
}

Inner y=new Inner();
y.innermethod();
```

```
        }

        public static void main(String args[])
        {
        MethodLocalInnerClasses x= new MethodLocalInnerClasses();
        x.outermethod();
        }
        }
```

### 3. Static Nested Class:

Static nested classes are not technically inner classes. They are like a static member of outer class.

**Example:**

```
// Java Program to Illustrate Static Nested Classes

// Importing required classes
import java.util.*;

class Outer {

   private static void outerMethod()
   {

      System.out.println("inside outerMethod");
   }

   static class Inner {

      public static void display()
      {

         System.out.println("inside inner class Method");

         outerMethod();
      }
   }
}

   public static void main(String args[])
   {

      // Calling method static display method rather than an instance of that class.
      Outer.Inner.display();
   }
```

## 4. Anonymous Inner Classes

Anonymous inner classes are declared without any name at all. They are created in two ways.

As a subclass of the specified type
As an implementer of the specified interface

Example:

```
package Inner_Class;

class Anonymous
{
   void show()
   {
      System.out.println("Anonymous show method");
   }
}

class Demo {
   static Anonymous d = new Anonymous()
   {
      void show() {
         super.show();
         System.out.println("Demo show method");
      }
   };

   public static void main(String[] args) {
      d.show();
   }
}
```

## ❖ Interface

The interface in Java is a mechanism to achieve abstraction(Hiding implementation details of code and show only necessary information to user). Traditionally, an interface could only have abstract methods (methods without a body) and public, static, and final variables by default. It is used to achieve abstraction and multiple inheritances in Java. In other words, interfaces primarily define methods that other classes must implement. Java Interface also represents the IS-A relationship.

Syntax for Java Interfaces

```
interface {
   // declare constant fields
   // declare methods that abstract
  }
```

Implementing an Interface:
A class implements an interface using the implements keyword.
The implementing class must provide concrete implementations for all the abstract methods declared in the interface.

**Example:**

```
import java.io.*;

// A simple interface
interface In1 {

   // public, static and final
   final int a = 10;

   // public and abstract
   void display();
}

// A class that implements the interface.
class TestClass implements In1 {

   // Implementing the capabilities of
   // interface.
   public void display(){
     System.out.println("Geek");
   }

   // Driver Code
   public static void main(String[] args)
   {
```

```
    TestClass t = new TestClass();
    t.display();
    System.out.println(t.a);
  }
}
```

## ❖ Inheritance

It is one of the Oops principles where one class acquires properties of another class with the help of 'extends' keywords is called Inheritance.
-The class from where properties are acquiring/inheriting is called
Super/ base/ parent class.
-The class to where properties are inherited/ delivered is called sub/child class.
-Inheritance takes place between 2 or more than 2 classes.

Example:

```
package inheritance_examples;

public class parent_class
{
        public void home()
        {
          System.out.println("Home");
        }
        public void car()
        {
          System.out.println("Car");
        }
        public void money()
        {
          System.out.println("Money");
        }
}

package inheritance_examples;
public class Child_class extends parent_class
{
        public static void main(String[] args)
        {
          parent_class obj= new parent_class();
          obj.home();
          Child_class obj1= new Child_class();
          obj1.home();
        }}
```

**Types of inheritance:**

**1. Single Inheritance/simple/singlelevel**

- It is an operation where inheritance takes place between only 2 classes.
-To perform single-level inheritance only 2 classes are mandatory.
-If only one base class is used to derive only one subclass, then it is referred as single-level inheritance or if only one sub class acquires property of one superclass, then it is refereed as single level inheritance.

Example:

```
package inheritance_examples;

public class father
{
	public static void home()
	{
		System.out.println("Parent has Banglow");
	}

	public void farm()
	{
		System.out.println("Parent has 4 Acres farm");
	}
}
package inheritance_examples;

class son extends father
{
//	public static void home()
//	{
//		System.out.println("Parent has Banglow");
//	}
//
//	public void farm()
//	{
//		System.out.println("Parent has 4 Acres farm");
//	}
```

**Above comments are related to methods which are inherited from parent class**

```
	public void mobile()
	{
		System.out.println("Son has Realme mobile");
	}

	public static void main(String[] args)
```

```
	{
		son obj= new son();

		obj.farm();

		home();

		obj.mobile();
	}
}
```

**2. Multilevel Inheritance:**

- Multilevel Inheritance takes place between 3 or more than 3 classes.
- In Multilevel Inheritance 1 sub class acquires properties of another super class & that class acquires properties of its another super class & phenomenon continuous.
- In the Multilevel inheritance, a derived class will inherit a base class and as well as the derived class also act as the base class to other class.

Example:

```
package inheritance_examples;
public class Paytm_v1
{
	public void moneyTransfer()
	{
		System.out.println("Money transfer feature implemented");
	}
}
package inheritance_examples;

 class Paytm_v2 extends Paytm_v1
{
//	public void moneyTransfer()
//	{
//		System.out.println("Money transfer feature implemented");
//	}

	public void flightbooking()
	{
		System.out.println("Flight booking feature introduced");
	}
}
package inheritance_examples;

class Paytm_v3 extends Paytm_v2
{
```

```
//	public void moneyTransfer()
//	{
//		System.out.println("Money transfer feature implemented");
//	}
//
//	public void flightbooking()
//	{
//		System.out.println("Flight booking feature introduced");
//	}

	public static void electricitybiilpayment()
	{
		System.out.println("Electricity bill payment feature introduced");
	}
}

package inheritance_examples;

class Paytm_v4 extends Paytm_v3
{

	public static void fastag()
	{
		System.out.println("Fastag feature introduced");
	}

	public static void main(String[] args)
	{
		Paytm_v4 obj= new Paytm_v4();

		obj.flightbooking();

		obj.moneyTransfer();

		electricitybiilpayment();

		fastag();
	}
}
```

**3. Hierarchical Inheritance**

- When multiple sub classes can acquire properties of 1 super class is known as hierarchical inheritance.

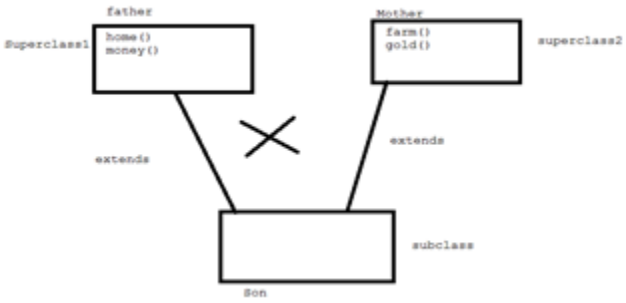
Example:

```
package Inheritance;

public class BaseClass {

	public void openbrowser()
	{
		System.out.println("browser opened");
	}

	public void openApp()
	{
		System.out.println("App opened");
	}
	public void Login()
	{
		System.out.println("App opened");
	}
}

 class Mobile extends BaseClass
 {
	public void testmobile()
	{
		System.out.println("test mobile successfully");
	}
 }

 class Myprofile extends BaseClass
 {
	 public void profilecheck()
	 {
		 System.out.println("profile check successfully");
	 }
	 public static void main(String args[])
	 {
		 Myprofile m= new Myprofile();
		 m.openbrowser();
		 m.openApp();
		 m.Login();
		 m.profilecheck();
```

```
                Mobile m1= new Mobile();
                m1.testmobile();
        }
}
```

**4. Multiple Inheritance**

- If one sub class acquiring properties of two super class at the same time then it is referred as Multiple Inheritance
- Java doesn't support Multiple inheritance using class because of "Diamond Ambiguity" problem.




Talking about Multiple inheritance is when a child class is inherits the properties from more than one parents and the methods for the parents are same (Method name and parameters are exactly the same) then child gets confused about which method will be called. This problem in Java is called the Diamond problem.

Example:

```
// Java Program to demonstrate Diamond Problem
package Inheritance;

public class MultipleEx {

        public void display()
        {
                System.out.println("display method called");
        }
}

 class MultipleEx2
 {
         public void display()
                {
                        System.out.println("display1 method called");
                }
 }
```

```
class Test extends MultipleEx,MultipleEx2
{
        public static void main(String[] args)
        {
                Test t= new Test();
                t.display();
        }
}
```

In above example we have seen that “test” class is extending two classes “MultipleEx2” and “MultipleEx” and its calling function “display()” which is defined in both parent classes so now here it got confused which definition it should inherit.

Multiple inheritances can be achieved using interface.

## ❖ Inheritance with constructor:

- If inheritance is performed between classes and you create object of child class and call constructor then it will first call constructor from parent/superclass and then it will call constructor of child class.

Example:

Constructor without parameter in superclass and Constructor without parameter in subclass

```
package Inheritance;

public class Inheritancewithconstructor {

        Inheritancewithconstructor()
        {
                System.out.println("Inheritancewithconstructor constructor executed");
        }

}


class Department extends Inheritancewithconstructor{
        Department()
   {
   System.out.println("Student constructor executed");
   }
```

```
}

class Student extends Department
{

  Student()
   {
   System.out.println("Student constructor executed");
   }

  public static void main(String[] args)
  {
          new Student();
  }
}
```

## ❖ Abstract Class

- A class declared with “abstract” keyword is called abstract class.
- An Abstract class is nothing but an incomplete class where programmer can declare complete as well as incomplete methods in it. (It requires Min 1 Complete and 1 Incomplete Method)
- We can't create object of abstract class to create object of abstract class we need to make use of concrete class.

**Points to Remember in Abstract:**

- If we declare a class as abstract then it is not necessary to declare an abstract method
- But if we declare any one of the method as abstract then that class must be declare as abstract class
- We cannot create an **Object** of abstract class
- But We can still access the complete methods from abstract class by creating an object of fully implemented child class (Concept of overriding)
- We cannot make static method as abstract reason abstract is based on overriding concept and **static method cannot be overridden**
- Abstract class extension **will work inside package only** , if we want abstract class accessibility throughout the project we need to use **public abstract**

Example:

```
package Abstarction;

public abstract class Simcard {
        abstract void sms();
        abstract void calls();
```

```
	abstract void internetuse();
	abstract void callertune();

}

class Jio extends Simcard
{


	void sms() {
		System.out.println("100sms per day");

	}


	void calls() {

		System.out.println("unlimited National calls");
	}


	void internetuse() {

		System.out.println("2gb per day");
	}


	void callertune() {

		System.out.println("Jio caller tune");
	}

}

class Airtel extends Simcard
{
	void sms()
	{
		System.out.println("200sms per day");
	}
	void calls()
	{
		System.out.println("unlimited National and international calls");
	}
	void internetuse()
	{
```

```
            System.out.println("3gb per day");
        }
        void callertune()
        {
            System.out.println("airtel caller tune");
        }

        public static void main(String [] args)
        {
            Jio j =new Jio();
            System.out.println("Jio sim card details here---->");
            j.callertune();
            j.internetuse();
            j.sms();

            Airtel a= new Airtel();
            System.out.println("\nAirtel sim card details here---->");
            a.calls();
            a.callertune();
            a.sms();
        }

}
```

## ❖ Final Class:

If we declare any class as final then we cannot inherit that class.

Syntax:

```
final class classname
{

}
```

**Example:**

```
class A
{
   final void m1()
   {
      System.out.println("This is a final method.");
   }
}
class B extends A
{
   void m1()
   {
```

```
        // Compile-error! We can not override
        System.out.println("Illegal!");
    }
}
```

❖ **Difference Between Abstract class and Final Class**

| Abstract | Final |
|---|---|
| Uses the “abstract” key word. | Uses the “final” key word. |
| This helps to achieve abstraction. | This helps to restrict other classes from accessing its properties and methods. |
| For later use, all the abstract methods should be overridden | Overriding concept does not arise as final class cannot be inherited |
| A few methods can be implemented and a few cannot | All methods should have implementation |
| Abstract class methods functionality can be altered in subclass | Final class methods should be used as it is by other classes |
| Cannot create immutable objects (infact, no objects can be created) | Immutable objects can be created (eg. String class) |

## ❖ Object wrapper and autoboxing

In Java, object wrappers are classes that encapsulate primitive data types, allowing you to treat primitive values as objects. These wrapper classes provide useful methods and enable you to work with primitive types in contexts where objects are expected, such as collections (e.g., ArrayLists) and when using Java's generics.
Object wrappers also allow you to represent nullable values, as they can hold a null reference.
Here are the common object wrapper classes for primitive data types:

- Integer for int
- Long for long
- Float for float

- Double for double
- Character for char
- Boolean for boolean
- Byte for byte
- Short for short

Let's look at an example using the Integer wrapper class:

```
// Using Integer wrapper class
Integer num1 = new Integer(42); // Creating an Integer object
Integer num2 = 10; // Autoboxing: Primitive int to Integer
System.out.println("num1: " + num1); // Output: num1: 42
System.out.println("num2: " + num2); // Output: num2: 10
// Performing operations with Integer objects
int sum = num1 + num2; // Unboxing: Integer to int
System.out.println("Sum: " + sum); // Output: Sum: 52
// Converting an Integer object to a primitive int
int intValue = num1.intValue();
System.out.println("intValue: " + intValue); // Output: intValue: 42
// Converting a String to an Integer
String str = "12345";
Integer parsedInt = Integer.parseInt(str);
System.out.println("parsedInt: " + parsedInt); // Output: parsedInt: 12345
```

In this example:

- We create Integer objects num1 and num2. num2 demonstrates autoboxing, where a primitive int value is automatically converted to an Integer object.
- We perform arithmetic operations using Integer objects, which involves unboxing, where Integer objects are automatically converted to primitive int values.
- We retrieve the primitive int value from an Integer object using the intValue() method.
- We convert a String to an Integer using the parseInt method.

This example shows how Integer wrapper objects can be used to work with primitive int values in a more object-oriented way. Similar operations can be performed with other wrapper classes for different primitive types.

## ❖ Inheritance and interfaces:

Inheritance and interfaces are two fundamental concepts in Java's object-oriented programming (OOP) that allow you to create and organize classes in a hierarchical and modular way.

**Combining Inheritance and Interfaces:**
You can also combine inheritance and interfaces to create complex class hierarchies and ensure that classes implement specific behaviors. For example:

```
// Superclass
class Animal {
void eat() {
System.out.println("The animal eats.");
}
}
// Interface
interface Swimmer {
void swim();
}
// Subclass implementing an interface
class Dolphin extends Animal implements Swimmer {
@Override
public void swim() {
System.out.println("The dolphin swims.");
}
}
public class Main {
public static void main(String[] args) {
Dolphin dolphin = new Dolphin();
dolphin.eat(); // Inherited from Animal class
dolphin.swim(); // Defined in Dolphin class
}
}
```

In this example, Dolphin is a subclass of Animal that also implements the Swimmer interface, allowing it to inherit the eat() method from Animal and provide its own swim() implementation.

| Inheritance | Interface |
|---|---|
| Inheritance is the mechanism in java by which one class is allowed to inherit the features of another class. | Interface is the blueprint of the class. It specifies what a class must do and not how. Like a class, an interface can have methods and variables, but the methods declared in an interface are by default abstract (only method signature, no body). |
| It is used to get the features of another class. | It is used to provide total abstraction |

| | |
|---|---|
| It is used to provide 4 types of inheritance. (multi-level, simple, hybrid and hierarchical inheritance) | It is used to provide 1 types of inheritance (multiple). |
| It uses extends keyword. | It uses implements keyword. |
| It overloads the system if we try to extend a lot of classes. | System is not overloaded, no matter how many classes we implement. |

## ❖ Introduction to packages

In Java, a package is a way to organize and structure your code into namespaces or directories. Packages provide several benefits, including code organization, encapsulation, and access control. They are essential for managing large-scale projects and avoiding naming conflicts. Here's an introduction to packages in Java:
Key Concepts:

1. Code Organization:
- Packages allow you to organize your code into logical groups or namespaces. This makes it easier to manage and maintain your codebase, especially in large projects.

2. Encapsulation:
- Packages provide a level of encapsulation. Classes and other types within a package are grouped together and can access each other's package-private (default) members. Outside code typically cannot access package-private members.

3. Access Control:
- Java provides access modifiers like public, private, protected, and package-private (no modifier) to control access to classes, methods, and variables. Classes and members within the same package can often access each other without the need for explicit access modifiers.

4. Naming Conventions:
- Package names are typically written in reverse domain name notation (e.g., com.example.myapp). This helps avoid naming conflicts across different projects.

5. Import Statements:
- To use classes or other types from a different package, you can use import statements in your Java code. Import statements specify the fully qualified class names or specific classes to be used from other packages.

**Package Declaration:**

To declare that a Java class belongs to a specific package, you include a package declaration at the beginning of the Java source file. The package declaration specifies the package name. For example:

```
package com.example.myapp;
public class MyClass {
// Class members and methods here }
```

Importing Packages and Classes:
To use classes from other packages or classes within your own package, you can use import statements. Import statements inform the compiler about the classes you want to use.
For example:

```
import java.util.ArrayList;
import java.util.List;
public class MyApp {
public static void main(String[] args) {
List<String> myList = new ArrayList<>();
// ...
}
}
```

In this example, we import the ArrayList and List classes from the java.util package.
Package Directory Structure:
In Java, the package name corresponds to a directory structure. For example, the package com.example.myapp would be represented as the directory structure com/example/myapp. Each directory contains the source files for the classes within that package.

**Benefits of Using Packages:**

- Code Organization: Packages help you organize your code logically, making it easier to navigate and maintain.
- Access Control: Package-private access control (no modifier) allows you to restrict access to certain classes or members within the package.
- Avoiding Naming Conflicts: Using unique package names (usually based on domain names) helps prevent naming conflicts, especially when integrating with libraries or other projects.
- Encapsulation: Classes within a package can access each other's package-private members, facilitating encapsulation.